Overview of Computer Architecture

Presented By

Dr. Priyanka Chakraborty

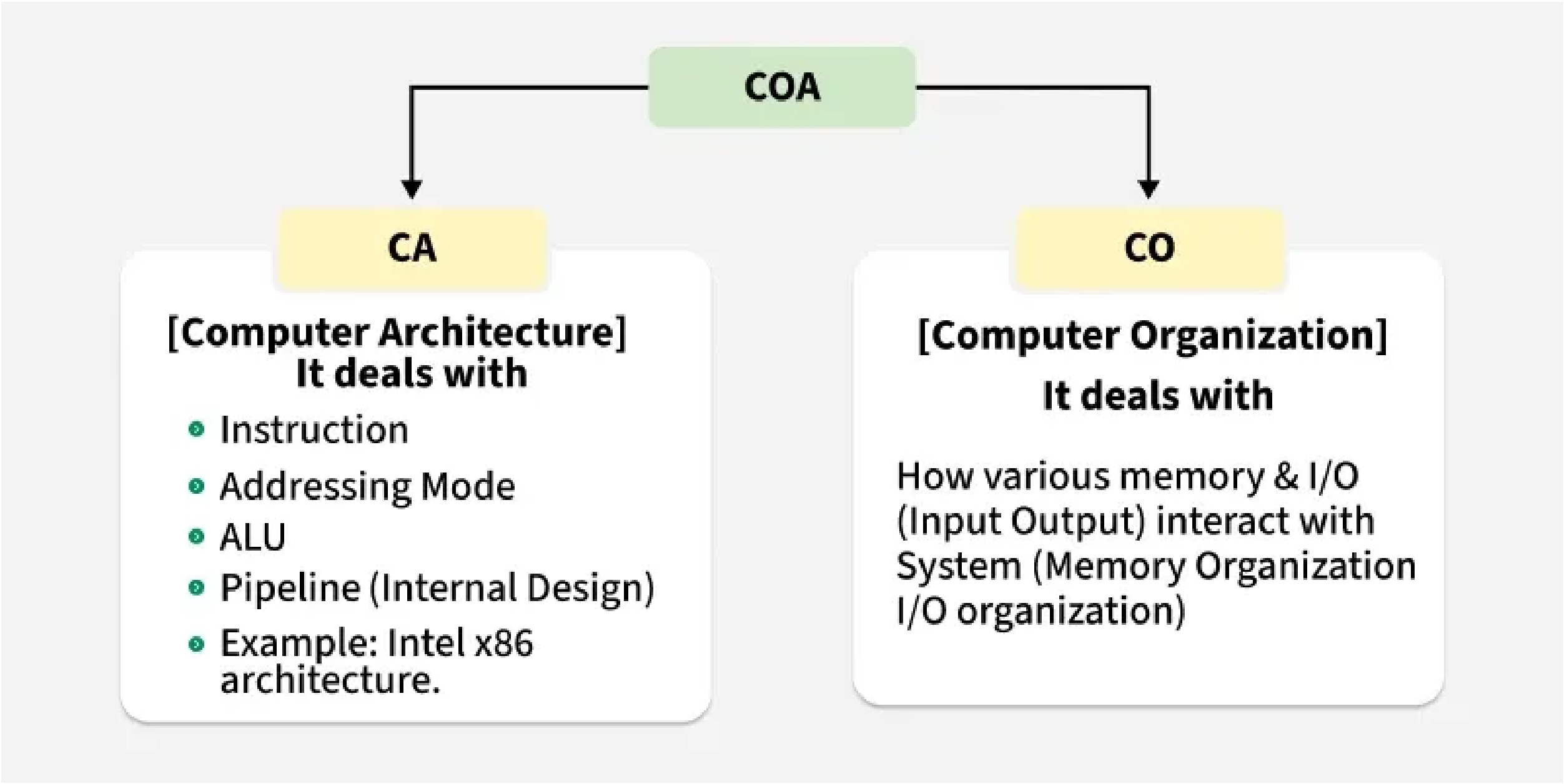
Introduction

Computer Architecture (CA)

- **Computer Architecture** describes **what a computer system *does*** and **how it appears to software** — especially the programmer. It defines the *functional behaviour* and *rules* of a computer system.
- **Key points**
  - ✓ Focuses on **instruction set**, **data types**, **addressing modes**, registers, and I/O mechanisms.
  - ✓ Deals with **logical design** and **high-level structure** that affects how software runs.
  - ✓ It is the part that is **visible to programmers** (e.g., what instructions you can use).
  - ✓ Basically answers: **"What should a computer do?"**
- **Examples:**
- Instruction set (like x86, ARM)
- Number of general-purpose registers
- Supported data formats and addressing modes

Computer Organization (CO)

- **Computer Organization** focuses on **how the computer's components are *physically implemented*** to achieve the architecture. It deals with the *hardware structure* and *control signals* that make the computer work.


COA
CA
[Computer Architecture]
It deals with
Instruction
Addressing Mode
ALU
Pipeline (Internal Design)
Example: Intel x86 architecture.
CO
[Computer Organization]
It deals with
How various memory & I/O (Input Output) interact with System (Memory Organization I/O organization)

Feature	**Computer Architecture (CA)**	**Computer Organization (CO)**
Focus	**What** the computer does	**How** the computer does it
Level	High-level design/specification	Low-level hardware implementation
Visibility	Visible to programmer	Hidden from programmer
Deals with	ISA, registers, data types	Hardware circuits, control signals
Example questions	“What instructions are available?”	“How do we implement these instructions?”

Functional blocks of a computer

1. Input Unit

Purpose: Captures data and instructions from users or external sources.

Function: Converts user input into signals that the computer can process.

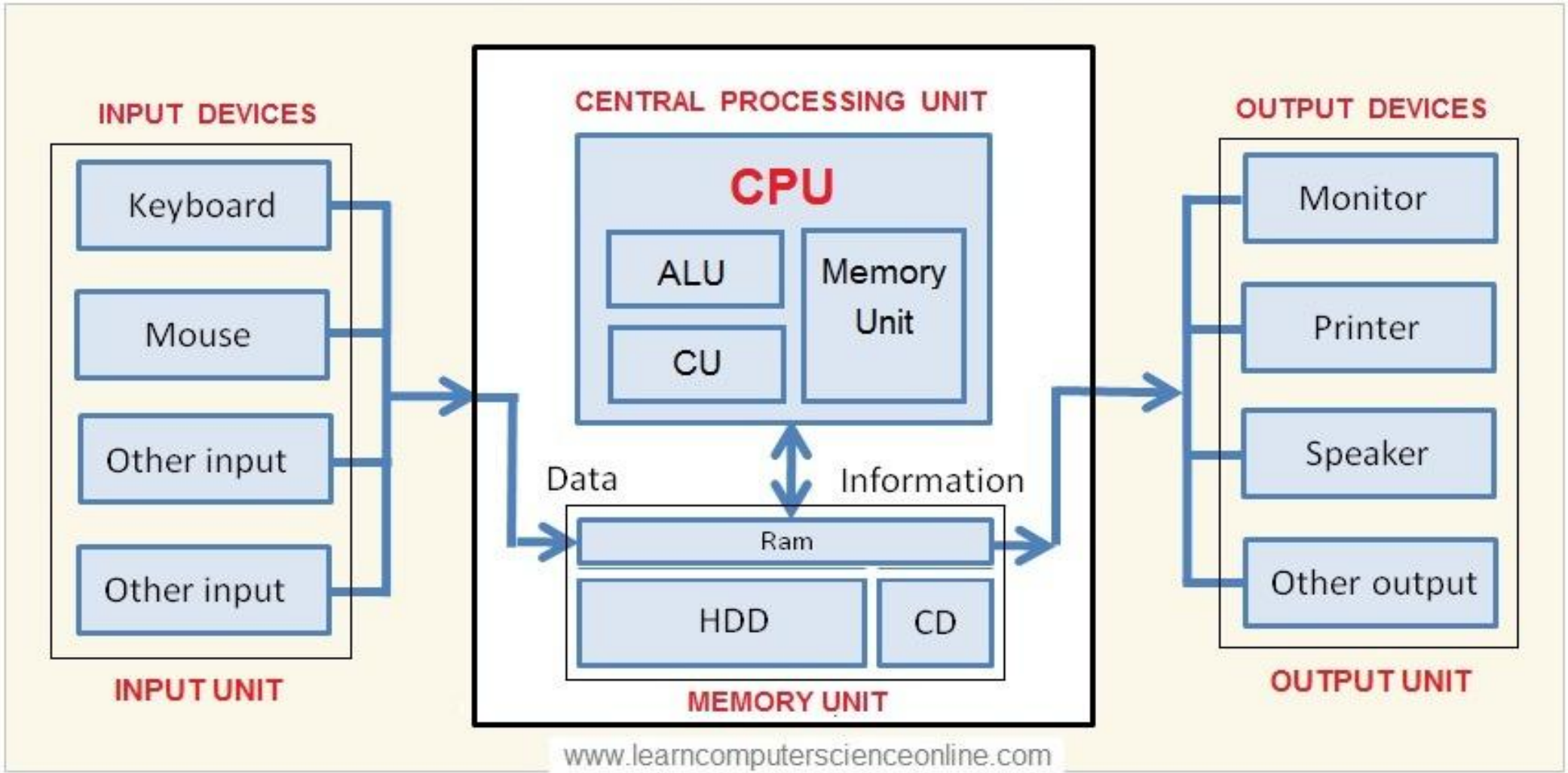
Common Devices (2025):

Keyboard, Mouse, Touchscreens
Scanners, Sensors, Stylus pens
Voice Assistants (e.g., Siri, Alexa)
Biometric devices (face/fingerprint recognition)
Iot-based inputs from smart devices

2. Central Processing Unit (CPU) – The Brain of the Computer

The **CPU** executes instructions and controls all internal operations. In 2025, CPUs will often have multiple **cores** and **threads** to handle parallel processing efficiently.

Computer System Block Diagram



Components of CPU:

a. Arithmetic Logic Unit (ALU)

Performs arithmetic operations (add, subtract, multiply, divide).

Handles logical operations (comparison, decision-making).

Supports AI/ML tasks using built-in vector/matrix operations (in modern CPUs).

b. Control Unit (CU)

- The **Control Unit (CU)** is a part of the **CPU (Central Processing Unit)** that directs and manages all operations inside the computer.
- It acts like a **manager or supervisor** of the CPU.

What the Control Unit Does

- **Fetches instructions** from memory
- **Decodes instructions** (understands what action is required)
- It receives external instructions or commands, which it converts to a sequence of control signals.
- **Directs other components** like: ALU (Arithmetic Logic Unit), Registers, Memory, Input/Output devices
- **Controls data flow** between CPU, memory, and peripherals

How It Works (Step-by-Step)

- During the **Fetch–Decode–Execute cycle**:
- **Fetch** → Gets instruction from RAM
- **Decode** → Interprets the instruction (to understand the opcode)
- **Send Signals** → Tells ALU or other units what to do by generating the control signal
- **Control Execution** → Ensures correct execution order of the instructions.

Important Features of Control Unit

- Does **not perform calculations** (ALU does that)
- Generates **control signals**
- Maintains proper **sequence of instructions**
- Ensures proper timing and coordination

Simple Example

- If the instruction is: ADD A + B
- Control Unit fetches the instruction
- Decodes that it is an addition
- Sends signal to ALU
- ALU performs addition
- Result stored in register

Types of Control Unit

- **Hardwired Control Unit**
  - Uses fixed electronic circuits
  - Faster
  - Difficult to modify
- **Microprogrammed Control Unit**
  - Uses microinstructions stored in memory
  - Easier to modify
  - Slightly slower

- **Hardwired Control Unit**
- In a Hardwired Control Unit, fixed hardware logic generates control signals based on the instruction's opcode.
- **Micro Programmable control unit**
- In microprogrammed control units, subsequent instruction words are fetched into the instruction register in a normal way.

c. Registers

High-speed memory locations within the CPU.

Temporarily store instructions, addresses, and intermediate data.

Examples: Accumulator, Instruction Register, Program Counter, Address Register.

Modern CPUs include 64-bit or even 128-bit registers for faster processing.

3. Memory / Storage Unit

The **memory unit** holds data and instructions before, during, and after processing.

a. Primary Memory (Main Memory):

RAM (Random Access Memory): Temporarily stores data during execution.
Types in 2025: DDR5, LPDDR5X, and emerging **MRAM**.

ROM (Read-Only Memory): Stores boot-up instructions and firmware.

Cache Memory: Ultra-fast memory between CPU and RAM (L1, L2, L3 levels).

b. Secondary Storage:

Used for long-term data storage.

Examples: SSDs (NVMe drives), HDDs, flash drives, and cloud storage.

Modern Trend: Use of **Cloud Integration** and **hybrid storage models**.

4. Output Unit

Purpose: Converts processed data (binary) into a form users can understand.

Examples:

Visual: Monitors (LED, OLED, 4K/8K displays)
Print: Printers (Inkjet, Laser, 3D Printers)
Audio: Speakers, Headphones
Haptic: Vibration feedback devices

Emerging Tech: AR/VR headsets, voice-based output, Braille displays for accessibility

Computer Bus structure

- A computer bus is a communication system within a computer or between computers that transfers data between different components. The purpose of buses is to reduce the number of "pathways" needed for communication between the components, by carrying out all communications over a single data channel.
- A bus is a set of physical connections (cables, circuits, etc.) that can be shared by multiple hardware components to communicate with one another. Memory and input/ output devices are connected to the Central Processing Unit through a group of lines called a bus. These lines are designed to transfer data between different components.
- **Types of Computer Bus**
- Address Bus
- Data Bus
- Control Bus

Computer Bus structure

1. Address Bus

A collection of wires used to identify particular location in main memory is called Address Bus. Or in other words, the information used to describe the memory locations travels along the address bus.

The address bus transports memory addresses which the processor wants to access in order to read or write data..

The address bus is unidirectional.

The size of address bus determines how many unique memory locations can be addressed.

Example:

A system with 4-bit address bus can address 24 = 16 Bytes of memory.
A system with 16-bit address bus can address 216 = 64 KB of memory
A system with 20-bit address bus can address 220 = 1 MB of memory.

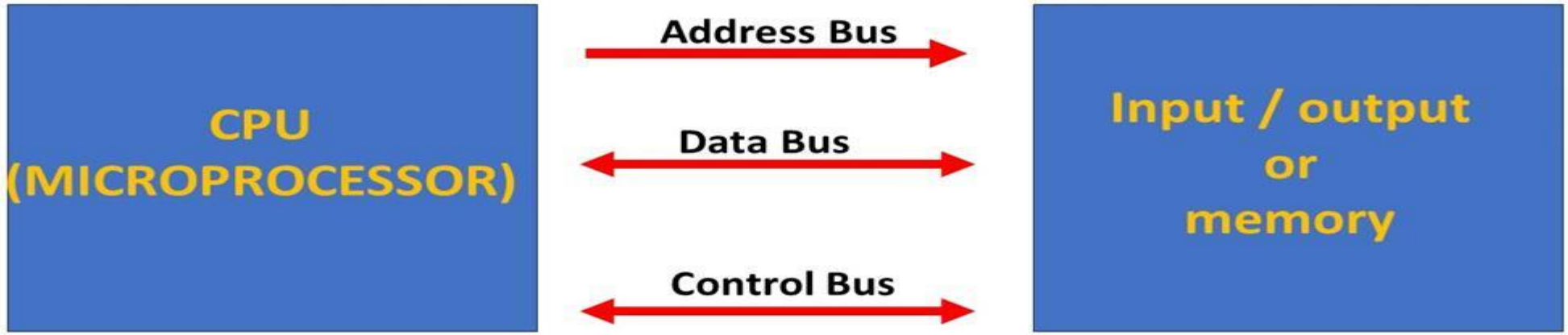
Computer Bus structure

2. Data Bus

- A collection of wires through which data is transmitted from one part of a computer to another is called Data Bus.
- Data Bus can be thought of as a highway on which data travels within a computer.
- The main objective of data bus is transfer of the data between microprocessor to input/ output devices or memory.
- The data bus transfers instructions coming from or going to the processor.
- The data bus is bidirectional because the data can flow in either direction from CPU to memory(or input/output device) or from memory to the CPU.
- The size (width) of bus determines how much data can be transmitted at one time.
- **Example:**
  - A 16-bit bus can transmit 16 bits of data at a time.
  - 32-bit bus can transmit 32 bits at a time.

3. Control Bus

- The connections that carry control information between the CPU and other devices within the computer is called Control Bus.
- The main objective of control bus is all signals controller carried from processor to other hardware device.
- The control bus transports **orders and synchronization signal coming from the control unit and travelling to all other hardware components**
- The Control bus is bidirectional because the signal can flow in either direction from CPU to memory(or input/output device) or from memory to the CPU.
- it also transmits response signals from the hardware.
- **Example:**
  - This bus is used to indicate whether the CPU is reading from memory or writing to memory.

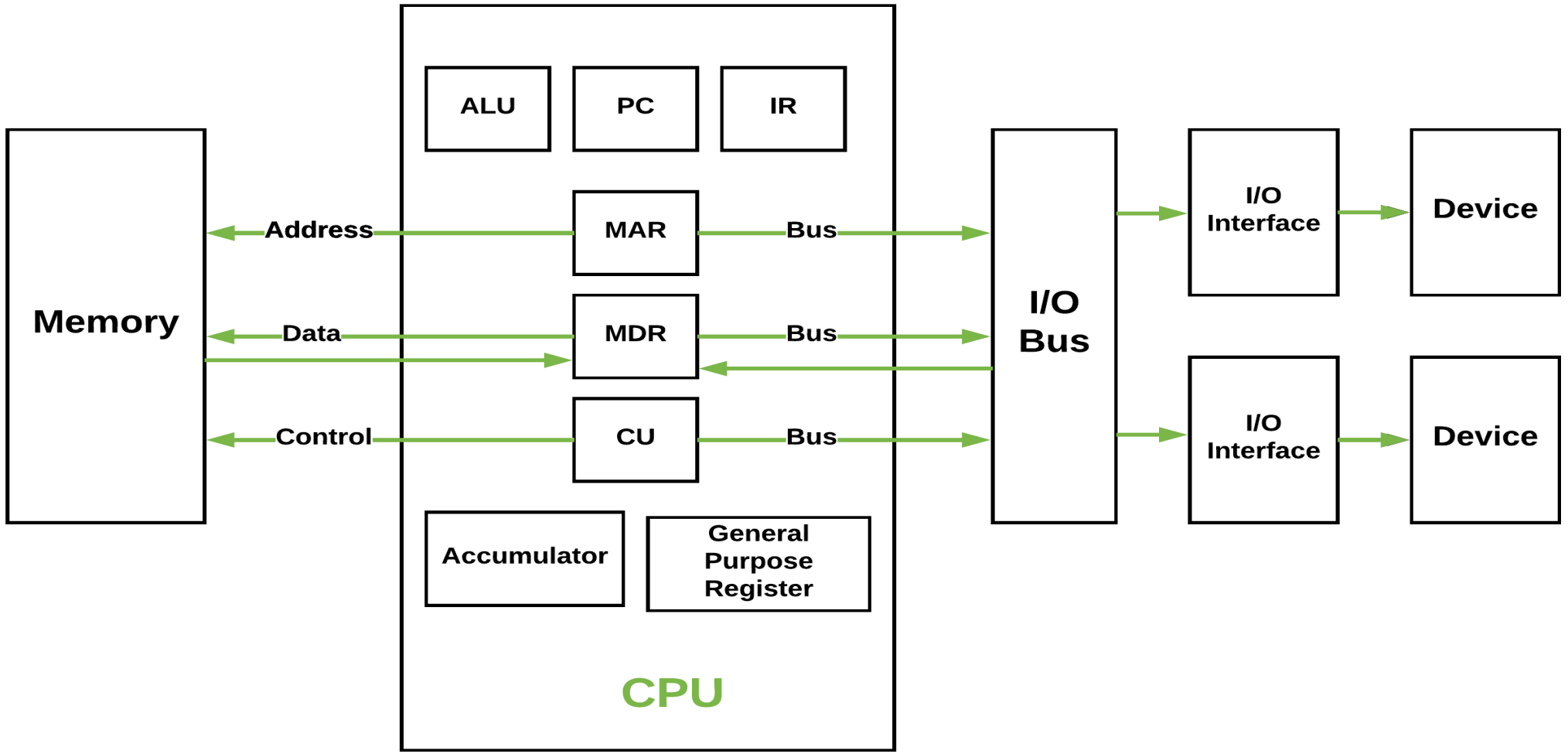


Types of Computer Architecture

- Computer architecture refers to the conceptual design and structure of a computer system, focusing on how the hardware and software interact.
- 1. **Von Neumann Architecture-**Shared memory for data and instructions.
- 2. **Harvard Architecture-**Separate memory for data and instructions.
- 3. **Modified Harvard Architecture-**Hybrid memory model.
- 4. **Instruction Set Architecture (ISA) Types-RISC vs CISC vs NISC** – Different instruction set philosophies.
- 5. **Flynn's Taxonomy — (SISD, SIMD, MISD, MIMD)** – Parallelism classification
- 6. **Multiprocessor & Parallel Architectures-** Parallel CPUs for high performance

1. Von Neumann Architecture

- The structure describes **Von Neumann Architecture**, which is a foundational design for modern computers. In this architecture, both **data and instructions are stored in the same memory and share a common bus for communication.** Here's an explanation of the components of Von Neumann architecture:



Von Neumann Architecture

- **Memory**
  - **Address**: Specifies the location in memory where data or instructions are stored or retrieved.
  - **Data**: The actual information (either data or instructions) stored in memory.
  - **Control**: Manages the flow of data and instructions between memory and the CPU
- **CPU (Central Processing Unit)**
  - The CPU is the core processing unit that executes instructions. It consists of:
  - **ALU (Arithmetic Logic Unit)**: Performs all **arithmetic** (addition, subtraction) and **logical** (AND, OR, comparisons) operations.
  - **PC (Program Counter)**: Keeps track of the address of the next instruction to be executed.
  - **IR (Instruction Register)**: Holds the current instruction being executed.
  - **MAR (Memory Address Register)**: Stores the address of the memory location being accessed.
  - **MDR (Memory Data Register)**: Temporarily holds data being transferred to or from memory.
  - **CU (Control Unit)**: Coordinates the activities of the CPU, managing the flow of data and instructions.
  - **Accumulator**: A register that stores intermediate results of arithmetic and logic operations.
  - **General Purpose Registers**: Used for temporary storage of data during processing.

Von Neumann Architecture

- **Bus**
  - The bus is a communication system that transfers data, addresses, and control signals between the CPU, memory, and I/O devices. In Von Neumann architecture, a single bus is shared for both data and instructions, which can create a bottleneck (known as the Von Neumann bottleneck).
- **I/O Bus**
  - **I/O Interface**: Connects the CPU and memory to input/output devices.
  - **Device**: Refers to external hardware like keyboards, monitors, or storage devices.
- **Key Characteristics of Von Neumann Architecture**
  - **Single Memory for Data and Instructions**: Both data and program instructions are stored in the same memory.
  - **Shared Bus**: A single bus is used for transferring data, addresses, and control signals, which can limit performance.
  - **Sequential Execution**: Instructions are executed one at a time in a sequential manner.

Von Neumann Architecture

➢ **Advantages of Von Neumann Architecture**

- **Simplified Design**: Uses a single memory for data and instructions, reducing hardware complexity.
- **Cost-Effective**: Lower production costs due to fewer components.
- **Flexibility**: Can run various programs and makes it suitable for general-purpose computing.
- **Ease of Programming**: Unified memory structure simplifies software development.
- **Widely Adopted**: Forms the foundation of most modern computers hence, ensures widespread compatibility.

➢ **Limitations of Von Neumann Architecture**

- **Memory Bottleneck**: Shared memory slows down data and instruction transfer.
- **Sequential Processing**: Cannot process data and instructions simultaneously.
- **Scalability Issues**: Struggles with high-performance tasks requiring rapid memory access.
- **Energy Inefficiency**: Frequent memory access increases power consumption.
- **Latency**: Data and instruction fetch delays reduce overall system efficiency.

➢ **Applications of Von Neumann Architecture**

- **General-Purpose Computing**: Powers desktops, laptops, and smartphones.
- **Embedded Systems**: Used in simple devices where cost and simplicity are priorities.
- **Software Development**: Shapes programming tools and languages due to its unified structure.
- **Education**: A foundational concept in computer science courses.
- **Gaming and Multimedia**: Supports complex applications like video games and editing software.

2. Harvard Architecture

In a normal computer that follows Von Neumann architecture, **instructions and data** both are stored in the same memory.

So same buses are used to fetch instructions and data.

Consequently, the CPU cannot do both things together (read the instruction and read/write data). So, to overcome this problem, Harvard architecture was introduced.

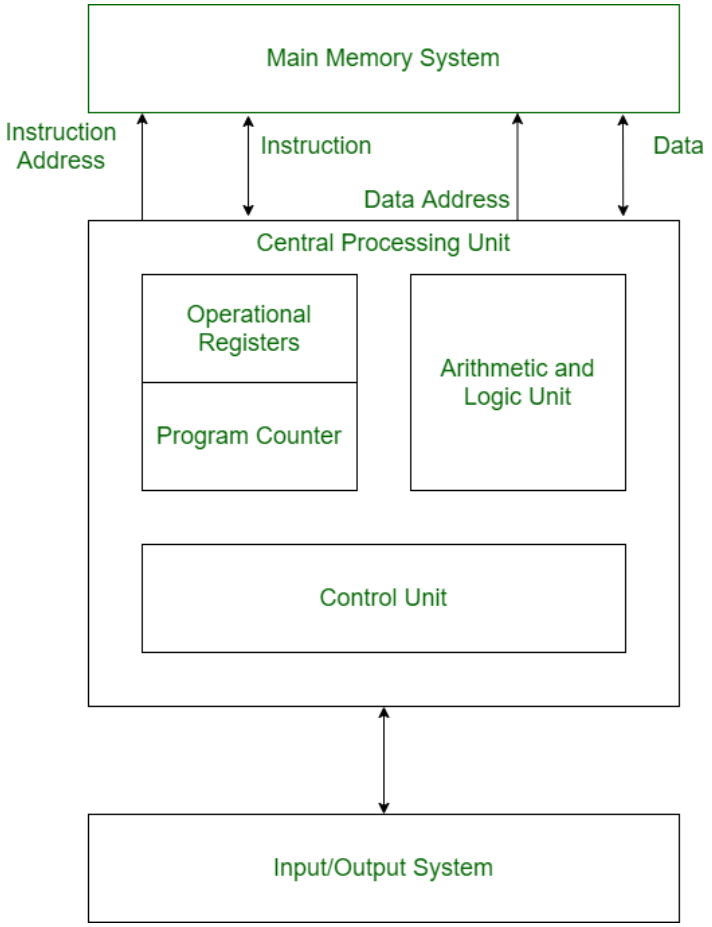
- The Harvard architecture is a computer architecture that **separates memory storage and buses** for instructions and data, unlike the von Neumann architecture, which uses a single memory and bus for both.
- This separation allows the CPU to **access instructions and read/write data simultaneously**, overcoming the bottleneck of von Neumann's architecture and **enhancing processing speed.**
- By enabling **parallel access to instructions and data,** the Harvard architecture improves performance, making it particularly suitable for applications requiring **high-speed processing**, such as **embedded systems and digital signal processing.**

Harvard Architecture

Buses

Buses are used as signal pathways. In Harvard architecture, there are separate buses for both instruction and data. Types of Buses:

- **Data Bus:** It carries data among the main memory system, processor, and I/O devices.
- **Data Address Bus:** It carries the **address of data** from the processor to the main memory system.
- **Instruction Bus:** It carries instructions among the main memory system, processor, and I/O devices.
- **Instruction Address Bus:** It carries the **address of instructions** from the processor to the main memory system.



Harvard Architecture

➢ **Features of Harvard Architecture**

- **Separate memory spaces:** Harvard architecture uses distinct memory spaces for instructions and data, enabling simultaneous access for faster processing.
- **Fixed instruction length:** In Harvard architecture, instructions are of fixed length, which simplifies the instruction fetch process and allows for faster instruction processing.
- **Parallel instruction and data access:** The separation of instruction and data memories allows parallel access, improving overall efficiency.
- **More efficient memory usage:** Harvard architecture allows for more efficient use of memory as the data and instruction memories can be optimized independently, which can lead to better performance.
- **Suitable for embedded systems:** Harvard architecture is commonly used in embedded systems because it provides **fast and efficient access to both instructions and data**, which is critical in real-time applications.
- **Limited flexibility:** The separate memory spaces restrict tasks like modifying instructions at runtime, reducing flexibility.

Harvard Architecture

➢ Advantage of Harvard Architecture

- **Fast and efficient data access:** Since Harvard architecture has separate memory spaces for instructions and data, it allows for parallel and simultaneous access to both memory spaces, which leads to faster and more efficient data access.
- **Better performance:** The use of fixed instruction length, parallel processing, and optimized memory usage in Harvard architecture can lead to improved performance and faster execution of instructions.
- **Suitable for real-time applications:** Harvard architecture is commonly used in embedded systems and other real-time applications where speed and efficiency are critical.
- **Security:** The separate spaces can also provide a degree of security against certain types of attacks, such as buffer overflow attacks.

➢ Disadvantages of Harvard Architecture

- **Complexity:** The use of separate memory spaces for instructions and data in Harvard architecture adds to the complexity of the processor design and can increase the cost of manufacturing.
- **Limited flexibility:** Harvard architecture has limited flexibility in terms of modifying instructions at runtime because instructions and data are stored in separate memory spaces. This can make certain types of programming more difficult or impossible to implement.
- **Higher memory requirements:** Harvard architecture requires more memory than Von Neumann architecture , which can lead to higher costs and power consumption.
- **Code size limitations:** Fixed instruction length in Harvard architecture can limit the size of code that can be executed, making it unsuitable for some applications with larger code bases.

3. Modified Harvard Architecture

- A **Modified Harvard Architecture** is a variation of the traditional **Harvard architecture** that *loosens the strict separation between instruction and data memory* while still keeping many of the performance benefits of separate access paths. It is very common in modern CPUs, caches, microcontrollers, and embedded processors.
- The classic **Harvard architecture** strictly separates instruction memory and data memory, with independent buses so the CPU can *fetch an instruction and access data at the same time* — removing the bottleneck seen in Von Neumann systems.
- A **Modified Harvard Architecture** retains this *separate access* capability but modifies how memory is organised or accessed, making the design more flexible and closer to real-world usage.

Types of Modifications

- Typical ways a Harvard architecture is modified include:
- **Split-cache Modified Harvard**
  Separate *instruction cache* and *data cache*, but a unified main memory. The CPU still sees separate caches but a single address space overall.
- **Instruction-Memory-as-Data**
  Allows program code or constant data in instruction memory to be *read as data*, useful for embedded constants without duplicating them.
- **Data-Memory-as-Instruction**
  Some processors can execute instructions fetched from any memory segment — not just a dedicated code memory — combining flexibility with parallel access.

4. Instruction Set Architecture (ISA) Types

- ■ **RISC (Reduced Instruction Set Computer)**
- Uses a **simplified instruction set** where most instructions are simple and quick.
- Emphasizes efficiency and pipelining.
- Common in mobile and embedded CPUs (like many ARM (Advanced Risc machine) chips).
- ■ **CISC (Complex Instruction Set Computer)**
- Contains **more complex instructions**, where single instructions may perform multiple operations.
- Used in many traditional desktop and server CPUs (like x86).

(i) RISC — Reduced Instruction Set Computer

- **Philosophy:**
  RISC processors use a *small, simple set of instructions* that do basic operations and typically execute in **one clock cycle**.
- **Key Characteristics**
- Simple, **uniform instruction format** (often fixed length).
- Emphasis on **register-to-register operations** with a *load/store* model.
- Easy to pipeline — overlapping instruction execution stages for speed.
- Compilers play a bigger role in optimizing instruction sequences.
- **Advantages**
- ✓ Faster execution per instruction due to simplicity
  ✓ Lower power consumption (ideal for mobile/embedded)
  ✓ Simplified hardware design and efficient pipelining
- **Drawbacks**
- Often needs **more instructions** to perform complex tasks
  Programs may be larger (more memory used) compared to CISC
- **Examples:** ARM, RISC-V, MIPS (RISC-type ISAs)

(ii). CISC — Complex Instruction Set Computer

- **Philosophy:**
  CISC architectures include a **large and rich set of instructions**, some capable of performing multiple low-level operations (like load + compute + store) in *one instruction*.
- **Key Characteristics**
- **Complex, variable-length instructions**.
- Can perform multi-step operations within a single instruction.
- More diverse addressing modes and fewer general-purpose registers.
- Often uses *microcode* to implement complicated instructions internally.
- **Advantages**
- ✓ Smaller code size — fewer instructions per program
  ✓ Supports legacy and backward-compatible instruction sets
  ✓ Instructions can be powerful — potentially fewer code fetches
- **Drawbacks**
- ✕ Harder to decode and pipeline due to instruction complexity
  ✕ More hardware resources needed — higher power and silicon cost
- **Examples:** Intel x86 and AMD x86-64 processors (CISC ISAs)

Feature	RISC	CISC
Instruction Set	Small, simple	Large, complex
Instruction Length	*Usually* fixed	Variable
Execution	Often 1 cycle	Multi-cycle
Decoder	Simple	Complex
Compiler Role	High	Moderate
Pipeline Friendly	Strong	Challenging
Code Size	Usually larger	Smaller
Use Case	Mobile, embedded, high-perf	Desktops, legacy systems
Real-world Examples	ARM, RISC-V	x86

- **RISC** prioritises *simplicity and efficiency* with a reduced instruction set and predictable execution.
- **CISC** aims to *reduce program size* and encode complex operations directly, trading off decoding complexity.
- **NISC** challenges traditional ISA design by removing the instruction set *entirely* and letting the compiler drive hardware control signals directly.

5. Parallelism–Based Architectures (Flynn's Taxonomy)

- Flynn's Taxonomy classifies architectures based on the number of **instruction streams** and **data streams** they handle simultaneously:
- **7. SISD (Single Instruction, Single Data)**
- One instruction stream working on one data stream.
- Traditional sequential computer model.
- **8. SIMD (Single Instruction, Multiple Data)**
- One instruction operates on multiple data elements in parallel.
- Common in vector processing and graphics (e.g., GPUs).
- **9. MISD (Multiple Instruction, Single Data)**
- Multiple instruction streams operate on the same data.
- Rare in practice; sometimes seen in specialised or fault-tolerant systems.
- **10. MIMD (Multiple Instruction, Multiple Data)**
- Multiple processors execute different instructions on different data.
- Used in multi-core processors, parallel systems, clusters, and distributed computing.

Multiprocessor & Parallel Architectures

- **Multiprocessor and parallel architectures** are system designs that use **more than one processing unit** (CPU/core) to perform computations *simultaneously* rather than sequentially. Their main goal is to **boost performance, throughput, and responsiveness**, especially for large, compute-intensive tasks like scientific simulation, big data analysis, graphics rendering, and high performance computing (HPC).
- A **multiprocessor** system has *multiple CPUs* working together within a single computer system, sharing resources like memory and I/O.
- **Parallel architecture** refers to how these processors (or cores) are organised and work together to solve problems faster by dividing work across units.
- **Why Use Multiprocessors & Parallel Architectures?**
- ✓ **Higher throughput:** Multiple processors can execute many tasks at once, greatly increasing computational speed.
  ✓ **Improved reliability:** If one processor fails, others can continue working.
  ✓ **Better scalability:** Adding more processors scales performance as demands grow.
  ✓ **Better multitasking:** Multiple independent tasks can be run concurrently without waiting for one to finish.
- **Example uses:** supercomputers, data centre servers, scientific simulations, large-scale databases, AI training systems, graphics rendering farms, and advanced engineering workloads.